

Booleani

Booleani

I valori vero/falso in Python — la base di ogni decisione nel codice.

Cos'è un booleano?

Nella vita di tutti i giorni facciamo continuamente valutazioni del tipo "vero o falso":

- La luce è accesa? Sì o No.
- Sono maggiorenne? Vero o Falso.
- Fa freddo fuori? Vero o Falso.

In Python esistono esattamente questi due valori: **True** (vero) e **False** (falso). Il tipo di dato che li contiene si chiama **booleano** (`bool`).

Attenzione: la prima lettera è **sempre maiuscola**. `true` e `false` (tutto minuscolo) causano un errore.

```
luce_accesa = True
fa_freddo = False
print(type(luce_accesa))  # <class 'bool'>
```

I confronti producono booleani

Ogni volta che confronti due valori, Python restituisce **True** o **False** :

```
print(5 > 3)      # True  - 5 è maggiore di 3? Sì
print(5 < 3)      # False - 5 è minore di 3? No
print(5 == 5)     # True  - 5 è uguale a 5? Sì
print(5 != 3)     # True  - 5 è diverso da 3? Sì
print(5 >= 5)     # True  - 5 è maggiore o uguale a 5? Sì
print(5 <= 4)     # False - 5 è minore o uguale a 4? No
```

:::caution `=` e `==` sono cose diverse!

- `=` assegna un valore a una variabile: `x = 5`
- `==` confronta due valori: `x == 5` (è uguale a 5?) :::

Combinare condizioni con `and`, `or`, `not`

Spesso vuoi combinare più condizioni insieme. In Python si usano tre operatori logici.

`and` — "e"

Restituisce `True` solo se **tutte** le condizioni sono vere. Come il semaforo verde: puoi passare solo se la strada è libera E il semaforo è verde.

```
eta = 20
ha_patente = True

# Puoi guidare solo se sei maggiorenne E hai la patente
print(eta >= 18 and ha_patente) # True
```

`or` — "o"

Restituisce `True` se **almeno una** condizione è vera. Come entrare in un cinema: puoi entrare se hai il biglietto O sei un membro del club.

```
ha_biglietto = False
e_membro = True

print(ha_biglietto or e_membro) # True — basta una delle due
```

`not` — "non"

Inverte il valore: trasforma `True` in `False` e viceversa.

```
piove = False
print(not piove)    # True – NON piove, quindi posso uscire

maggiorenne = True
print(not maggiorenne)  # False
```

Valori "truthy" e "falsy"

In Python, non solo `True` e `False` possono essere trattati come booleani. **Ogni valore** può essere interpretato come vero o falso.

I valori che Python considera **falsi** (equivalenti a `False`):

- `False`
- `0` e `0.0`
- `""` (stringa vuota, cioè nessun testo)
- `[]` (lista vuota)
- `None` (assenza di valore)

Tutto il resto è considerato **vero**. Un numero diverso da zero, una stringa con del testo, una lista con elementi.

```
print(bool(0))        # False – zero è falso
print(bool(""))       # False – stringa vuota è falsa
print(bool([]))       # False – lista vuota è falsa

print(bool(42))       # True  – qualsiasi numero non-zero è vero
print(bool("ciao"))    # True  – qualsiasi testo non-vuoto è vero
print(bool([1, 2]))    # True  – una lista con elementi è vera
```

Questo torna molto utile nelle condizioni. Per esempio, puoi controllare se un nome è stato inserito così:

```
nome = ""
if nome:
    print("Ciao, " + nome + "!")
else:
    print("Non hai inserito nessun nome.") # questa viene stampata
```